

▲ Pyramidal Hash 256×512

- > A novel cryptographic hash function based on a **non-linear pyramidal reduction architecture**,
- > combining AES S-box confusion, position-dependent bit rotations, and π -derived round constants
- > inside a **sponge construction** with a 512-bit internal state and 256-bit output.

⚠ Security Notice

This is a **research prototype**. It has not been formally audited by the cryptographic community.

Do not use in production systems, security applications, or cryptographic protocols without independent peer review and formal cryptanalysis.

Overview

Property	Value
Output size	256 bits (32 bytes)
Internal state	512 bits (64 bytes)
Block size	256 bytes per absorption
Permutation rounds	12
S-box	AES S-box (NL=112)
Round constants	16 × 32-bit values from π
Architecture	Sponge construction
Implementation	Pure Python 3

How It Works

1. Pyramidal Nonlinear Function

The core component is a **pyramidal reduction**: a 64-byte vector is iteratively compressed level by level. At each level, adjacent bytes are combined using:

...

```
diff = SBOX[(seq[i+1] - seq[i]) mod 256]
diff = diff XOR anchor
```

```
diff = rotate_left(diff, (i mod 7) + 1)
...
```

This continues until a single byte remains (the "tip" of the pyramid).
The first element of each level forms the 64-byte output block.

2. Internal Permutation (12 rounds)

Each round applies three steps to the full 512-bit state:

1. **Pyramidal mixing** — each 64-byte chunk is processed with the pyramidal function and XOR-combined with position- and round-dependent constants.
2. **Linear diffusion** — each byte is XOR-ed with the next, spreading influence across the state.
3. **Nonlinear substitution** — AES S-box applied to every byte.

3. Padding & Absorption

Padding follows a SHA-like scheme:

- Append ``0x80``
- Zero-pad until ``len(buffer) mod 256 == 248``
- Append the message length in bits as a 64-bit little-endian integer

Installation

No external dependencies required. Pure Python 3.6+.

```
```bash
git clone https://github.com/your-username/pyramidal-hash.git
cd pyramidal-hash
python3 pyramidal_hash.py
```
```

Usage

```
```python
from pyramidal_hash import PyramidalHash

Basic usage
h = PyramidalHash()
```

```

h.update(b"Hello, world!")
print(h.hexdigest(32)) # 256-bit output (32 bytes hex)

String input
h2 = PyramidalHash()
h2.update("Hello, world!") # UTF-8 encoded automatically
digest = h2.digest(32) # Returns bytes

Streaming (large data)
h3 = PyramidalHash()
h3.update(b"first chunk")
h3.update(b"second chunk")
print(h3.hexdigest(32))

Configurable output length
h4 = PyramidalHash()
h4.update(b"test")
print(h4.hexdigest(64)) # 512-bit output
...

```

#### ### Example outputs

```

...
Input: b"" → a44748f9b8879a0ee3bb9940ea4adf5e...
Input: b"\x00" → fe544bdeb9314cb9e408446f3a17aa84...
Input: b"\x01" → 48b44e3c031e10f98df2bf6a5fba09ae...
Input: b"hola" → e365f529838ee87eed46ff0726dab7a5...
Input: b"Hola" → e7540a07e109ff52bd36e7fb837ed8ee...
Input: b"password" → 87044325187a44ad91fe2ecca352fffb...
...

```

Every input — including an empty string vs. a single `\x00`` byte — produces a completely different hash. A single space or capital letter changes the entire output.

---

#### ## Statistical Test Results

All tests performed with cryptographically random inputs (``secrets.token_bytes``).

#### ### Avalanche Effect (300 samples)

Metric	Result	Ideal	Status
-----	-----	-----	-----

Average avalanche	**49.90%**	50.00%		
Standard deviation	**3.05%**	< 5%		
Minimum	38.67%	—		
Maximum	58.20%	—		

Flipping a single input bit changes, on average, exactly half the output bits — statistically indistinguishable from ideal behavior.

#### ### Uniformity — Chi-Square Test (19,200 bytes)

Metric	Result	Critical value (95%)	Status	
-----	-----	-----	-----	
$\chi^2$ statistic	**270.96**	293.2		
Degrees of freedom	255	—		

Output bytes are uniformly distributed over [0, 255] at the 99% confidence level.

#### ### Pearson Correlation (400 samples, 496 pairs)

Metric	Result	Threshold	Status	
-----	-----	-----	-----	
Average correlation	**0.039**	< 0.05		
Maximum correlation detected	0.165	< 0.10		
Pairs with correlation > 0.10	21 / 496	0 (ideal)		

> The 21 pairs above 0.10 are likely statistical noise at N=400.

> Confirmation with  $N \geq 2,000$  samples is recommended.

#### ### Determinism (200 trials)

Metric	Result	Status	
-----	-----	-----	
Identical hashes	200 / 200		

#### ### Shannon Entropy

Metric	Result	Maximum	Status	
-----	-----	-----	-----	
Entropy	**> 7.99 b/B**	8.0 b/B		

---

#### ## Performance

> Measured on pure Python 3 — not representative of a compiled implementation.

Input size	Time / hash
64 bytes	~121 ms
1 KB	~305 ms
10 KB	~2,138 ms

SHA-256 processes the same inputs in **microseconds** in C.  
A C or Rust implementation of this design is planned as future work.

---

### ## Comparison with SHA-256

Property	Pyramidal Hash 256×512	SHA-256
Output bits	256	256
Internal state bits	512	256
Avalanche average	49.90%	~50.01%
Chi-square (19K B)	270.96 ✓	< 293.2 ✓
Formal audit	✗ None yet	✓ Extensive
Production ready	✗ Research only	✓ Yes
Speed (Python)	~121 ms / 64B	< 1 ms / 64B

---

### ## Design Decisions

Component	Choice	Rationale
S-box differential	AES S-box (unmodified)	Extensively studied; NL=112, minimal
Round constants	Digits of $\pi$	Nothing-up-my-sleeve construction; no hidden backdoors
Architecture (SHA-3)	Sponge construction	Formal security guarantees; proven paradigm
State size	512 bits	256-bit birthday bound security margin
Rounds	12	Conservative; comparable to Keccak-f[800]

---

### ## Known Limitations

- **\*\*No formal cryptanalysis\*\***: differential, linear, algebraic, and length-extension attacks have not been studied.
- **\*\*Correlation (minor)\*\***: 21 of 496 byte pairs show correlation > 0.10 at N=400 samples. Requires confirmation at larger sample sizes.
- **\*\*Performance\*\***: Python-only implementation is unsuitable for production use.
- **\*\*No external audit\*\***: peer review by the cryptographic community is pending.

---

### ## Reproducing the Tests

```
```bash
# Run all statistical tests
python3 analisis_profundo.py

# Quick functional test
python3 -c "
from pyramidal_hash import PyramidalHash
h = PyramidalHash()
h.update(b'test')
print('Hash:', h.hexdigest(32))
"
```
```

---

### ## Academic Paper

A full technical paper describing the design, implementation, and evaluation is available:

 **\*\*Pyramidal Hash 256×512: Design, Analysis and Evaluation of a Novel Cryptographic Hash Function\*\***  
 \_(submitted for peer review — link will be added upon publication)\_

---

### ## Contributing

Contributions are welcome, especially:

-  Formal cryptanalysis (differential, linear, algebraic attacks)
-  C / Rust / Go implementation for performance benchmarking
-  Additional statistical tests (NIST SP 800-22 test suite)
-  Mathematical proof of avalanche properties

Please open an issue or pull request.

---

## ## License

MIT License — see [LICENSE](LICENSE) for details.

---

## ## References

1. Daemen, J. & Rijmen, V. (2002). \*The Design of Rijndael: AES\*. Springer.
2. Bertoni, G. et al. (2011). \*Cryptographic sponge functions\*. IACR.
3. NIST (2015). \*SHA-3 Standard\*. FIPS PUB 202.
4. NIST (2012). \*SHA-2 Standard\*. FIPS PUB 180-4.
5. Menezes, A. et al. (1996). \*Handbook of Applied Cryptography\*. CRC Press.

---

<div align="center">

<sub>Pyramidal Hash 256×512 — Research prototype · Not for production use</sub>  
</div>